Atelier gRPC

1. Créer un nouveau projet :
    a. Name: *grpcServer*
    b. *Language: java*
    c. *Build System: Maven*
2. Ajouter les dépendances et les plugins : https://github.com/badrhr/grpcClient
3. Créer un fichier 'Bank.proto' dans le repertoire resources.

```
syntax = "proto3";
option java_package = "org.xproce.stubs" ;
service BankService{
  rpc convert(ConvertCurrencyRequest) returns (ConvertCurrencyResponse);
  rpc getCurrencyStream(ConvertCurrencyRequest) returns (stream ConvertCurrencyResponse);
  rpc performStream(stream ConvertCurrencyRequest) returns ( ConvertCurrencyResponse);
  rpc fullCurrencyStream(stream ConvertCurrencyRequest) returns ( stream ConvertCurrencyResponse);
}
message ConvertCurrencyRequest{
  string currencyFrom = 1 ;
  double amount = 2 ;
  string currencyTo = 3 ;
}
message ConvertCurrencyResponse{
  string currencyFrom = 1 ;
  double amount = 2 ;
  string currencyTo = 3 ;
  double result = 4 ;
}
```

4. Dans Maven → Lifecycle, sélectionner compile, en suite cliquer sur run.
5. Vérifier la création des deux classes : Bank et BankServiceGrpc dans le package *org.xproce.stubs*.
6. Créer la classe BankGrpcService dans le package *org.xproce.service*.
7. Etendre *BankServiceGrpc.BankServiceImplBase* et redéfinie la méthode *convert*.

```
import io.grpc.stub.StreamObserver;
import org.xproce.stubs.Bank;
import org.xproce.stubs.BankServiceGrpc;

public class BankGrpcService extends BankServiceGrpc.BankServiceImplBase {

    @Override
    public void convert(Bank.ConvertCurrencyRequest request,
StreamObserver<Bank.ConvertCurrencyResponse> responseObserver) {
        String currencyFrom  = request.getCurrencyFrom();
        String currencyTo  = request.getCurrencyTo();
        double amount  = request.getAmount();
        Bank.ConvertCurrencyResponse response = Bank.ConvertCurrencyResponse.newBuilder()
                                                .setCurrencyFrom(currencyFrom)
                                                .setCurrencyTo(currencyTo)
                                                .setAmount(amount)
                                                .setResult(amount * 12.14)
                                                .build();
        responseObserver.onNext(response);
        responseObserver.onCompleted();
    }
```

8. Créer la classe *GRPCServer* dans le package *org.xproce.server*.

```
package org.xproce.server;
import io.grpc.Server;
import io.grpc.ServerBuilder;
import org.xproce.service.BankGrpcService;
import java.io.IOException;
public class GrpcServer {
    public static void main(String[] args) throws IOException, InterruptedException {
        Server server = ServerBuilder.forPort(5555)
                                .addService(new BankGrpcService())
                                .build();
        server.start();
        server.awaitTermination();
    }
}
```

9. Créer un nouveau projet :
    a. Name: *grpcClient,*
    b. *Language: java,*
    c. *Build System: Maven.*
10. Répéter les étapes 2, 3, 4, et 5 dans le nouveau projet.
11. Créer une classe *BankGrpcFirstClient :*

```
package org.xproce.client;
import io.grpc.ManagedChannel;
import io.grpc.ManagedChannelBuilder;
import org.xproce.stubs.Bank;
import org.xproce.stubs.BankServiceGrpc;

public class BankGrpcFirstClient {
    public static void main(String[] args) {
        ManagedChannel managedChannel = ManagedChannelBuilder.forAddress("localhost",5555)
                                                .usePlaintext()
                                                .build();
        BankServiceGrpc.BankServiceBlockingStub blockingStub =
BankServiceGrpc.newBlockingStub(managedChannel);
        Bank.ConvertCurrencyRequest request = Bank.ConvertCurrencyRequest.newBuilder()
                                        .setCurrencyFrom("MAD")
                                        .setCurrencyTo("Euro")
                                        .setAmount(15425)
                                        .build();
        Bank.ConvertCurrencyResponse response = blockingStub.convert(request);
        System.out.println(response);
    }
}
```

12. Lancer le serveur créé dans la classe *GRPCServer.*
13. Tester le client BankGrpcFirstClient.